

# Auditoria de segurança — painel administrativo (admsite)

sosconsumidor.com.br · 25 de agosto de 2026

Escopo: `public_html/admsite/` (painel administrativo do site, 667 arquivos `.php` próprios, fora de bibliotecas de terceiros). Complementa a auditoria de estrutura/código de 19/08/2026, que cobriu principalmente a raiz do site público e o fluxo de login. Esta auditoria foi pedida especificamente para revisar o restante do painel administrativo.

## Resultado executivo

Esta auditoria encontrou uma superfície de risco bem maior do que a de 19/08: o painel administrativo é composto por cerca de 30 módulos com o mesmo padrão legado repetido (SQL por concatenação, escape inconsistente, controle de acesso decorativo), e pelo menos **dois achados eram exploráveis por qualquer visitante da internet sem nenhum login** — uma execução remota de código via upload de arquivo, e uma injeção SQL sem autenticação capaz de sobrescrever qualquer coluna de qualquer tabela (incluindo, por exemplo, a senha de uma conta de administrador). Por serem falhas ativas em produção, **ambas foram corrigidas de emergência durante esta auditoria**, com autorização do usuário, antes mesmo de terminar de escrever este relatório.

O restante dos achados (SQL injection mitigada apenas por escaping fraco, XSS armazenado em módulos com exposição pública indireta, controle de acesso por permissão desativado no código, sessão sem proteção contra fixação, bibliotecas desatualizadas) é sério, mas não estava sendo explorado ativamente no momento da auditoria — fica registrado como plano de correção priorizado, para execução em lotes, com testes, como já é o padrão deste projeto.

## Ações de emergência já executadas em produção (25/08/2026)

Com autorização do usuário, antes de concluir o restante do relatório:

### 1. Execução remota de código via upload de arquivo CORRIGIDO

Três endpoints de upload (`file_upload/server/php/arquivos.php`, `upload_arquivo/upload_multi_arquivos_processar.php`, `upload_arquivo/upload_arquivo_processar.php`) aceitavam qualquer extensão de arquivo sem validação de conteúdo, dois deles sem exigir login algum, salvando o arquivo dentro do webroot (`public_html/arquivos/`) — um visitante anônimo podia enviar um `shell.php` e executá-lo diretamente. Um quarto ponto, `upload_foto/upload_foto_processar.php` (usado pelo cadastro de banners), tinha a checagem de login **comentada** — a versão antiga preservada em `upload_foto/old/` confirma que isso é uma regressão recente, não um design original. Um quinto, `upload_foto/upload_foto_crop.php`, também sem login, permitia ler qualquer arquivo do sistema decodificável como imagem via um parâmetro `filename` sem sanitização (path traversal).

#### Correção aplicada:

- Novo `public_html/images/.htaccess` e `public_html/arquivos/.htaccess` (cobre subpastas por herança) — bloqueiam com `Require all denied` a execução de `.php` / `.phtml` / `.phar` / `.cgi` / `.pl` nesses diretórios, independente de qual script fez o upload ou de qual extensão o arquivo enviado tem. Testado: arquivo `.php` de teste retornou **403** em vez de

executar; imagens legítimas continuam servidas normalmente; site público e admsite continuam no ar.

- `upload_foto/upload_foto_processar.php` e `upload_foto/upload_foto_crop.php`: restaurada a exigência de login ( `Login::verificaToken()` ) — a versão anterior tentava chamar `Login::check()` , um método que **não existe** na classe `Login` . Testado: requisição sem token autenticado retorna **403**; funcionalidade para quem está logado não foi alterada.
- `upload_foto/upload_foto_crop.php` : parâmetro `filename` agora passa por `basename()` , fechando a leitura arbitrária de arquivo.

Os demais achados de upload (endpoints sem `checkAcao` granular, whitelist de extensão comentada, biblioteca de imagem de 2011, limite de tamanho quebrado) continuam **mitigados pelo bloqueio de execução acima**, mas ficam como P1 pendente de correção definitiva.

## 2. SQL injection sem autenticação via reordenação de itens CORRIGIDO

Quatro endpoints AJAX ( `perguntas_e_respostas/organizar_ajax.php` , `perguntas_e_respostas_areas/organizar_ajax.php` , `perguntas_e_respostas_grupo/organizar_ajax.php` , `banners/organizar_ajax.php` ) montavam um `UPDATE` de banco concatenando o **nome da tabela** e o **id** direto da requisição, sem validar nada e **sem exigir login**. Como o nome da tabela é colado cru na string SQL antes de `SET` , um visitante anônimo podia enviar um valor malicioso que reescrevesse a instrução inteira — por exemplo, sobrescrever a senha de uma conta de administrador diretamente, sem precisar de usuário/senha algum.

**Correção aplicada** nos 4 arquivos: passou a exigir login válido; o nome da tabela agora é validado contra uma lista fixa de um único valor permitido por módulo; os valores de id/ordem passam por conversão para número inteiro antes de entrar na consulta. Testado: requisição sem login retorna **403** nos 4 endpoints; conferido no banco que nenhuma tabela foi alterada pelos testes; funcionalidade de reordenar itens continua igual para quem está logado.

Backup dos 8 arquivos originais (antes de qualquer alteração) guardado no servidor. Cópia local sincronizada e conferida idêntica à produção nos 8 arquivos.

## 3. SQL injection sem nenhum escape em fóruns públicos, contato e notícias CORRIGIDO

Corrigidos, com backup e testes diretos contra o banco (inserindo tentativas de ataque reais e conferindo que ficam gravadas como texto puro, sem executar), os 8 pontos que gravavam dado de formulário público sem nenhuma proteção: a classe de gravação do formulário de contato foi reescrita para usar consultas parametrizadas; os métodos de gravação de pergunta/resposta dos dois fóruns públicos e de denúncia passaram a escapar texto e converter identificadores para número antes de gravar; um bug de variável trocada no cadastro de notícias, que descartava um filtro de segurança já pronto, foi corrigido; um bug idêntico na tela de visualização de código-fonte do informativo de newsletter (que descartava um valor já validado e usava o dado bruto da requisição) também foi corrigido, e essa mesma tela ganhou proteção contra código malicioso que faltava no título e no HTML exibidos; e a reordenação de notícias, que não exigia login algum, passou a exigir.

Durante os testes, uma primeira tentativa de correção no módulo de denúncias tratou um campo como texto; o teste direto contra o banco revelou que a coluna correspondente é na verdade numérica — o erro foi identificado e corrigido antes da publicação, graças ao próprio processo de teste.

#### 4. Controle de acesso, sessão e uploads CORRIGIDO

Antes de mexer no código, foi descoberto que ativar o bloqueio de permissão abaixo quebraria duas telas reais do painel ("Contato" e a tela ligada a "Trabalhe Conosco"), porque as duas contas de administrador existentes não tinham, no banco, o registro de permissão para essas telas específicas. Corrigido primeiro no banco de dados, testado, e só depois habilitado o bloqueio no código — para não trocar uma falha de segurança por uma tela quebrada.

- O bloqueio de permissão por tela, que estava desativado no código, agora funciona de verdade. Testado: as contas existentes continuam com acesso a tudo; uma conta sem permissão nenhuma é corretamente barrada.
- O parâmetro que escolhe qual tela abrir no painel agora só aceita um caminho que exista de fato dentro da pasta do painel. Testado com várias tentativas de "fugir" da pasta — todas rejeitadas; telas normais continuam abrindo normalmente.
- A sessão agora é renovada logo após o login, fechando uma brecha de sequestro de sessão.
- Os três endpoints de upload sem verificação de tipo de arquivo passaram a exigir login e a checar a extensão contra uma lista de tipos permitidos, além de sanitizar o nome do arquivo.
- Corrigido o erro de cálculo que permitia até 50 GB por upload (a mensagem dizia "500 MB"); esse endpoint também passou a exigir login.
- Nova proteção bloqueando acesso direto pelo navegador à pasta que concentra toda a lógica interna do painel — nada legítimo precisa acessá-la assim.

### Achados pendentes — P0 (graves, sem exploração pública ativa confirmada hoje)

#### Controle de acesso por permissão desativado no código

`admsite/classes/permissoao/Permissao.Class.php` (método `setSessao()`) checa se o nível do usuário logado tem permissão para uma tela, mas a linha que deveria bloquear o acesso está comentada:

```
if($checkSessao && !$this->checkSessao($sessaoId)){  
#    throw new ExceptionCustom("Você não tem permissão para esta sessão.");  
}
```

O menu lateral filtra corretamente o que aparece para cada nível — mas isso é só cosmético: o despachante principal só exige um login válido para abrir qualquer tela pedida pela URL, sem checar se o nível do usuário tem permissão para aquele módulo específico. Na prática, qualquer conta logada consegue visualizar qualquer tela de listagem/relatório do painel, mesmo que o menu nunca mostre esse link para o nível dela.

Ações de *escrita* (inserir/editar/excluir) são, na maior parte dos módulos, protegidas separadamente e funcionam corretamente. O problema é específico à *leitura* (visualização).

Impacto prático hoje: **baixo** — só existem 2 contas no admsite, ambas no nível mais alto, então não há hoje nenhuma conta de nível restrito para explorar essa falha. Mas precisa ser corrigida **antes** de criar qualquer conta de acesso limitado.

**Ação:** implementar de fato o bloqueio em `Permissao::setSessao()`, e testar com uma conta de nível restrito antes de confiar nisso.

#### Despachante do painel aceita qualquer caminho de arquivo

O parâmetro que escolhe qual tela abrir ( `?pg=` ) não filtra tentativas de "sair da pasta" (ex.: `../../../../`). Qualquer conta autenticada no admsite pode, em tese, forçar a inclusão de outro arquivo `.php` do servidor.

**Ação:** validar esse parâmetro contra uma lista de páginas conhecidas, rejeitando qualquer tentativa de navegação para fora da pasta do painel.

### SQL injection sem nenhum escape (dado alimentável por visitante anônimo)

Estas gravações usam concatenação de texto **sem nenhuma proteção**:

| Arquivo                                                                                  | Função                                                       | Origem do dado                                  |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------|
| <code>classes/formulario_contatos/FormularioContatos.Class.php</code>                    | <code>save()</code>                                          | Formulário público de contato/reserva           |
| <code>classes/forum_advogado/ForumAdvogado.Class.php</code>                              | <code>savePergunta()</code> ,<br><code>saveResposta()</code> | Fórum de advogados (público)                    |
| <code>classes/forum_consumidor/ForumConsumidor.Class.php</code>                          | <code>savePergunta()</code> ,<br><code>saveResposta()</code> | Fórum do consumidor (público)                   |
| <code>classes/forum_consumidor/ForunsDenuncias.Class.php</code>                          | <code>saveDenuncia()</code>                                  | Denúncia via fórum (público)                    |
| <code>admsite/noticias/db.php</code> +<br><code>NoticiasXEspecialidades.Class.php</code> | <code>save()</code>                                          | Bug de variável trocada — usa dado não validado |
| <code>admsite/noticias/organizar_ajax.php</code>                                         | reordenar notícias                                           | Sem exigir permissão de módulo                  |
| <code>admsite/newsletter/newsletter_ver_fonte.php</code>                                 | visualizar código-fonte                                      | Bug de variável trocada                         |
| <code>classes/noticias/Noticias.Class.php</code>                                         | <code>getSqlPesquisaAdm()</code>                             | Filtro de especialidades sem validação          |

**Ação:** migrar essas gravações/consultas para consultas parametrizadas (prepared statements) — são poucos pontos e concentram o risco mais alto do painel, por serem alimentados por formulários públicos sem exigir login.

**XSS armazenado-explorável por visitante anônimo — corrigido, ver item 5 das ações de emergência acima**

### Sessão sem proteção contra fixação de sessão

O código de login nunca regenera o identificador de sessão após confirmar a senha. Um atacante que consiga fixar uma sessão na vítima antes do login (por exemplo, por um link) herda a sessão já autenticada depois.

**Ação:** regenerar o identificador de sessão logo após confirmar a senha.

## **Conteúdo do editor de texto (TinyMCE) sem sanitização no servidor — corrigido, ver item 8 das ações abaixo**

### **Achados pendentes — P1**

---

- **SQL injection mitigada só por escape fraco, não por consultas parametrizadas:** padrão recorrente (mais de 30 pontos) em praticamente todas as telas de busca/filtro do painel. Não é uma injeção grave hoje, mas é uma proteção frágil espalhada por todo o sistema legado.
- **XSS armazenado sem exposição pública confirmada hoje, mas sem proteção:** listagens de consumidores, denúncias (fluxo parece desativado, mas dado antigo continua exibido) e usuários administrativos.
- **Biblioteca de envio de e-mail desatualizada** (SwiftMailer) — fim de vida desde novembro de 2021, sem correção disponível para uma vulnerabilidade conhecida na versão instalada. Ação: migrar para uma biblioteca mantida atualmente.
- **Editor de texto (TinyMCE) desatualizado** — versão anterior a correções de segurança conhecidas. Ação: atualizar para uma versão atual.
- **Falta de proteção de acesso direto na pasta de classes do sistema** — essa pasta concentra toda a lógica de negócio. Ação: bloquear acesso HTTP direto.
- **Falta de checagem de permissão específica nos endpoints de upload** — qualquer conta do admsite consegue fazer upload em qualquer módulo. Ação: aplicar a mesma checagem já usada em outras partes do sistema.
- **Limite de tamanho de upload quebrado** num dos processadores de galeria de fotos — a mensagem diz "500 MB" mas o código permite até 50 GB por um erro de cálculo, viabilizando esgotamento de espaço em disco. Ação: corrigir o valor.
- **Código de depuração esquecido em produção** em dois arquivos (um imprime a consulta SQL, outro imprime dados de upload sem autenticação). Ação: remover.

### **Achados pendentes — P2 / estrutura e limpeza**

---

- Arquivo de registro de erros antigo (2020/2021) na raiz do painel, expondo caminhos internos do servidor — mover para fora do site público ou apagar.
- Três cópias mortas de uma biblioteca de edição de texto (cerca de 820 arquivos, nenhuma referência ativa) — arquivar.
- Cópia duplicada e desatualizada do fluxo de upload de foto, ainda acessível diretamente pela internet — remover.
- Página de teste de uma biblioteca de terceiros, publicamente acessível, que faz upload/exclusão reais contra produção se alguém abrir — remover do site em produção.
- Classe de upload não utilizada, mas com configurações padrão perigosas (aceita qualquer tipo de arquivo) — remover para não ser reativada por engano no futuro.
- Arquivos de lixo do sistema operacional (Windows) guardados por engano dentro de pastas de bibliotecas antigas — remover.
- ~~Tempo de sessão por inatividade não é aplicado de fato~~ — corrigido (25/08), ver caixa abaixo.
- Pequenas exposições de baixo impacto (o painel reflete de volta parâmetros da própria navegação sem proteção, mas só afeta quem já está logado e clicar em um link malicioso). Corrigir por precaução, sem urgência.

## 5. XSS armazenado explorável por visitante anônimo CORRIGIDO

Corrigidos os 13 arquivos que exibiam, sem nenhuma proteção contra código malicioso, dados originados de formulários públicos (auto-cadastro de advogado, fale conosco/reserva/trabalhe conosco, opinião, fórum de advogados) ou de conteúdo externo curado (notícias, newsletter): a tela de edição do cadastro de advogado (14 campos), as telas de contato/reserva/trabalhe conosco, a listagem de opiniões, as telas do fórum de advogados, as telas de notícias e a montagem do e-mail de newsletter.

Testado: um trecho de código malicioso de teste é corretamente neutralizado (vira texto inofensivo em vez de executar), inclusive combinado com a função que converte quebra de linha em `  
>` (quebra de linha continua funcionando, código malicioso neutralizado). Site público, painel administrativo, notícias e cadastro de advogado testados e funcionando normalmente.

## Bug pré-existente encontrado durante o teste ao vivo e corrigido

Ao testar as telas do painel depois do lote de correções de controle de acesso, foi reportado que a tela "Consumidores" mostrava dados nas colunas erradas (telefone aparecia vazio, e-mail aparecia deslocado). Confirmado: bug antigo sem relação com as correções desta auditoria — a linha da tabela imprimia uma coluna extra (CPF) sem cabeçalho correspondente, empurrando todas as colunas seguintes uma posição para a direita, e a coluna "Data" nunca era preenchida. Corrigido; conferido direto no banco que os dados agora batem com o cabeçalho — o cadastro usado como exemplo realmente não tem telefone cadastrado, por isso aparecia em branco. A tela de Denúncias, que usa um padrão parecido, foi conferida e não tem o mesmo problema.

## 6. Tempo de sessão por inatividade + limpeza estrutural CORRIGIDO

O tempo de última atividade, que já existia gravado mas nunca era conferido, agora é checado a cada acesso ao painel: depois de 60 minutos parado, a sessão expira sozinha e pede login de novo (cada clique renova o prazo). Testado com sessões simuladas de diferentes tempos de inatividade — comportamento correto em todos os casos.

Arquivadas as cópias antigas do editor de texto, a cópia duplicada do fluxo de upload de foto, o arquivo de registro de erros antigo e a página de teste de terceiros; removidos os arquivos de lixo do sistema e a classe de upload morta com configuração perigosa; removida uma linha de depuração que expunha uma consulta SQL interna.

**Achado extra durante a limpeza:** um segundo arquivo de depuração não era código morto — estava ligado ao botão de inserir imagem do editor de texto, e a linha de depuração **quebrava essa função silenciosamente**. Corrigido; a função de inserir imagem volta a funcionar.

**Achado extra e corrigido, fora do painel administrativo:** a página pública de auto-cadastro de advogado apresentava erro fatal intermitente, deixando o formulário fora do ar para quem tentasse se cadastrar — causado por um link para uma funcionalidade de busca de advogados que foi descontinuada (bloqueada de propósito no servidor) sem que os links do formulário de cadastro tivessem sido atualizados. Corrigido em 2 arquivos; confirmado no registro do servidor que o erro parou de ocorrer.

## 7. Cadastro de advogados e Fórum de Advogados desativados temporariamente

**FEITO — pedido do usuário**

A pedido do usuário, o cadastro público de advogados e o Fórum de Advogados foram desativados temporariamente — sem uso previsto agora, mas podem ser necessários no futuro. O código de ambos permanece correto e intacto; a desativação foi feita só na porta de entrada, retornando código de acesso

negado para quem tentar acessar. Diferente das funcionalidades já descontinuadas de vez, aqui a reativação é simples e reversível a qualquer momento, sem mexer em código.

Conferido que nenhum link do site público aponta para essas duas funcionalidades hoje, que as telas do painel que gerenciam os cadastros já existentes continuam funcionando normalmente, e que o restante do site não foi afetado.

8. Sanitização de HTML do editor de texto **CORRIGIDO**

Instalada a biblioteca HTMLPurifier no projeto (com verificação de assinatura antes de instalar; nenhuma vulnerabilidade própria encontrada). Criada uma função que permite uma lista de formatações seguras (negrito, itálico, links, imagens, tabelas, títulos, citação, vídeo incorporado do YouTube/Vimeo) e remove tanto tags quanto atributos perigosos fora dessa lista — diferente do filtro anterior, que só olhava o nome da tag e deixava passar atributos como `onerror` ou links `javascript:` dentro de tags já permitidas.

Testado com o fluxo real de gravação: um texto com tentativas de ataque (roubo de cookie via script, imagem com evento malicioso, link com código embutido) foi gravado de verdade através do sistema — o resultado ficou limpo, com o ataque removido e a formatação legítima preservada. Vídeo incorporado do YouTube testado e continua funcionando. Site público, notícias, páginas institucionais e painel administrativo testados e funcionando normalmente.

Escopo não coberto nesta rodada

Por profundidade/tempo, não foram revisados linha a linha alguns módulos de baixo risco (galeria de fotos legada aparentemente não utilizada, consulta de CEP, consulta de cidades/estados) e os arquivos de suporte que só contêm JavaScript, sem processar dados no servidor.

Plano de correção priorizado

| # | Ação                                                                                                       | Esforço     | Status                    |
|---|------------------------------------------------------------------------------------------------------------|-------------|---------------------------|
| 1 | Execução remota de código via upload sem autenticação/validação                                            | —           | Feito (emergência, 25/08) |
| 2 | SQL injection sem autenticação na reordenação de itens (4 arquivos)                                        | —           | Feito (25/08)             |
| 3 | SQL injection sem nenhum escape (fóruns públicos, contato, notícias, newsletter)                           | Médio       | Feito (25/08)             |
| 4 | XSS armazenado alimentável por visitante anônimo (advogados, contato, opinião, fórum, notícias/newsletter) | Médio       | Feito (25/08)             |
| 5 | Restaurar o bloqueio de permissão desativado + proteger o parâmetro de navegação do painel                 | Baixo–Médio | Feito (25/08)             |
| 6 | Regenerar sessão após login                                                                                | Baixo       | Feito (25/08)             |
| 7 | Sanitização de HTML do editor de texto no servidor                                                         | Médio       | Feito (25/08)             |

|    |                                                                                          |                |                                                               |
|----|------------------------------------------------------------------------------------------|----------------|---------------------------------------------------------------|
| 8  | Consultas parametrizadas no restante das buscas/filtros (mais de 30 pontos)              | Alto (gradual) | Pendente                                                      |
| 9  | Autenticação/tipo de arquivo nos uploads restantes + corrigir limite de tamanho quebrado | Baixo          | <b>Feito (25/08)</b> — falta checagem de permissão por módulo |
| 10 | Bloquear acesso direto à pasta de classes do sistema                                     | Baixo          | <b>Feito (25/08)</b>                                          |
| 11 | Limpeza estrutural (bibliotecas antigas, arquivo de registro, código de depuração)       | Baixo          | <b>Feito (25/08)</b>                                          |
| 12 | Migrar biblioteca de e-mail; atualizar editor de texto                                   | Alto           | Pendente                                                      |
| 13 | Aplicar tempo de sessão por inatividade real                                             | Baixo          | <b>Feito (25/08)</b>                                          |

Itens 3–6 e 9–10 são o próximo lote natural (mesmo padrão de risco dos itens já corrigidos de emergência, mas sem exploração pública confirmada hoje — dá para agendar com backup e teste, sem pressa de minutos). Itens 7, 8 e 12 são projetos maiores, a fazer em lotes.

---

Documento gerado a partir da auditoria técnica de 25/08/2026. Registro técnico completo (com trechos de código e caminhos exatos de arquivo) disponível em DOCUMENTACAO/AUDITORIA\_ADMSITE\_2026-08-25.md, na pasta do projeto.